

1. Class apiResponse

```
class ApiResponse
{
    public static function success(mixed $data = null, string $message = 'OK', int $status = 200): JsonResponse
    {
        return response()->json([
            'success' => true,
            'message' => $message,
            'data' => $data,
        ], $status);
    }

    public static function created(mixed $data = null, string $message = 'Created'): JsonResponse
    {
        return self::success($data, $message, 201);
    }

    public static function error(string $message, int $status = 400, array $errors = []): JsonResponse
    {
        $payload = [
            'success' => false,
            'message' => $message,
        ];

        if ($errors !== []) {
            $payload['errors'] = $errors;
        }

        return response()->json($payload, $status);
    }

    public static function validation(array $errors, string $message = 'Data tidak valid'): JsonResponse
    {
        return self::error($message, 422, $errors);
    }
}
```

2. Class Traceability (Record Event)

```
class Traceability
{
    public static function recordEvent(
        HarvestBatch $batch,
        string $title,
        ?User $actor = null,
        array $metadata = [],
        ?string $actorLabel = null,
    ): HarvestBatchEvent {
        $proof = DummyBlockchainLedger::prepareEvent($batch, $title, $actor,
        $metadata, $actorLabel);

        return HarvestBatchEvent::create([
            'harvest_batch_id' => $batch->id,
            'title' => $title,
            'actor_user_id' => $actor?->id,
            'actor_label' => $actorLabel ?? $actor?->full_name,
            'event_at' => $proof['event_at'],
            'metadata' => $metadata,
            'previous_ledger_hash' => $proof['previous_ledger_hash'],
            'ledger_hash' => $proof['ledger_hash'],
            'ledger_height' => $proof['ledger_height'],
        ]);
    }
}
```

3. Function storeShipmentBatches

```
public function storeShipmentBatches(Request $request): JsonResponse
{
    $user = $request->user();
    $this->ensureRole($user, UserRole::Pengepul->value);

    $data = $request->validate([
        'destination_type' => ['required', Rule::in(ShipmentDestination::values())],
        'source_batch_codes' => ['nullable', 'array', 'min:1'],
        'source_batch_codes.*' => ['string', 'exists:harvest_batches,code'],
        'packaged_at' => ['nullable', 'date'],
        'warehouse_note' => ['nullable', 'string'],
    ]);

    $batches = HarvestBatch::query()
```

```
->whereIn('code', $data['source_batch_codes'] ?? [])
->get()
->keyBy('code');
```

```
$shipment = DB::transaction(function () use ($user, $data, $batches) {
    $shipment = CollectorShipment::create([
        'code' => CodeGenerator::shipment(),
        'collector_user_id' => $user->id,
        'destination_type' => $data['destination_type'],
        'total_weight_kg' => $batches->sum('quantity_kg'),
        'total_fruit_count' => $batches->sum('fruit_count') ?: 0,
        'packaged_at' => $data['packaged_at'] ?? now(),
        'warehouse_note' => $data['warehouse_note'] ?? null,
        'status' => ShipmentStatus::ReadyToShip->value,
    ]);
```

```
    foreach ($batches as $batch) {
        CollectorShipmentSource::create([
            'collector_shipment_id' => $shipment->id,
            'harvest_batch_id' => $batch->id,
            'source_code_snapshot' => $batch->code,
            'source_grade_snapshot' => $batch->grade,
            'source_weight_kg' => $batch->quantity_kg,
            'source_fruit_count' => $batch->fruit_count,
            'source_variety_snapshot' => $batch->variety,
        ]);
    }
}
```

```
    return $shipment;
});
foreach ($batches as $batch) {
    $batch->forceFill([
        'status' => HarvestBatchStatus::InDistribution->value,
    ]->save();
```

```
    Traceability::recordEvent($batch, 'Batch Masuk Shipment', $user, [
        'status' => HarvestBatchStatus::InDistribution->value,
        'shipment_code' => $shipment->code,
        'destination_type' => $shipment->destination_type,
    ]);
}
return ApiResponse::created([
    'code' => $shipment->code,
    'status' => $shipment->status,
], 'Shipment berhasil dibuat');
}
```

4. Function verifyBatch

```
public function verifyBatch(Request $request, string $code): JsonResponse
{
    $user = $request->user();
    $this->ensureRole($user, UserRole::Pengepul->value);

    $batch = HarvestBatch::query()
        ->where('code', $code)
        ->firstOrFail();

    $data = $request->validate([
        'received_quantity_kg' => ['required', 'numeric', 'min:0'],
        'received_fruit_count' => ['required', 'integer', 'min:0'],
        'grade_breakdown' => ['nullable', 'array'],
        'quality_notes' => ['nullable', 'string'],
        'verified_by' => ['nullable', 'string', 'max:150'],
    ]);

    DB::transaction(function () use ($batch, $data, $user): void {
        $batch->forceFill([
            'status' => HarvestBatchStatus::VerifiedByCollector->value,
            'received_quantity_kg' => $data['received_quantity_kg'],
            'received_fruit_count' => $data['received_fruit_count'],
            'quality_notes' => $data['quality_notes'] ?? null,
            'verified_by_user_id' => $user->id,
            'verified_at' => now(),
        ])->save();

        Traceability::recordEvent($batch, 'Batch Diverifikasi Pengepul', $user, [
            'status' => HarvestBatchStatus::VerifiedByCollector->value,
            'verified_by' => $data['verified_by'] ?? $user->full_name,
        ]);
    });

    return ApiResponse::success([
        'code' => $batch->code,
        'status' => HarvestBatchStatus::VerifiedByCollector->value,
    ], 'Batch terverifikasi');
}
```

5. Function storeProduct

```
public function storeProduct(Request $request): JsonResponse
{
    $user = $request->user();
    $this->ensureRole($user, UserRole::Ukm->value);
}
```

```

$data = $request->validate([
    'name' => ['required', 'string', 'max:150'],
    'price' => ['nullable', 'numeric', 'min:0'],
    'price_label' => ['nullable', 'string', 'max:50'],
    'stock_qty' => ['nullable', 'integer', 'min:0'],
    'stock_label' => ['nullable', 'string', 'max:50'],
    'status' => ['nullable', Rule::in(ProductStatus::values())],
    'source_codes' => ['nullable', 'array'],
    'source_codes.*' => ['string', 'exists:harvest_batches,code'],
]);

$product = DB::transaction(function () use ($user, $data) {
    $product = UmkmProduct::create([
        'code' => CodeGenerator::product(),
        'umkm_user_id' => $user->id,
        'name' => $data['name'],
        'price' => $this->resolvePrice($data),
        'stock_qty' => $this->resolveStock($data),
        'status' => $data['status'] ?? ProductStatus::Aktif->value,
    ]);

    foreach (($data['source_codes'] ?? []) as $sourceCode) {
        $batch = HarvestBatch::query()->where('code', $sourceCode)->first();
        if (!$batch) {
            continue;
        }

        UmkmProductSource::create([
            'umkm_product_id' => $product->id,
            'harvest_batch_id' => $batch->id,
            'source_code_snapshot' => $batch->code,
        ]);

        Traceability::recordEvent($batch, 'Source batch dipakai untuk produk UMKM',
        $user, [
            'product_code' => $product->code,
        ]);
    }

    return $product;
});

return ApiResponse::created(ContractFormatter::product($product), 'Produk
berhasil dibuat');
}

```

6. Function storeTransaction

```
public function storeTransaction(Request $request): JsonResponse
{
    $user = $request->user();
    $this->ensureRole($user, UserRole::Konsumen->value);

    $data = $request->validate([
        'umkm_product_id' => ['nullable', 'integer', 'exists:umkm_products,id'],
        'quantity' => ['required', 'integer', 'min:1'],
        'total_amount' => ['nullable', 'numeric', 'min:0'],
        'total_label' => ['nullable', 'string', 'max:50'],
        'payment_status' => ['nullable', 'string', 'max:50'],
        'note' => ['nullable', 'string'],
    ]);

    $productId = $data['umkm_product_id'] ?? null;
    if (! $productId) {
        return ApiResponse::validation([
            'product_id' => ['Produk wajib dipilih.'],
        ]);
    }

    $product = UmkmProduct::query()->whereKey($productId)->firstOrFail();

    if (($product->stock_qty ?? 0) < ($data['quantity'] ?? 0)) {
        return ApiResponse::validation([
            'quantity' => ['Stok produk tidak mencukupi.'],
        ]);
    }

    $transaction = DB::transaction(function () use ($user, $data, $product) {
        $transaction = ConsumerTransaction::create([
            'code' => CodeGenerator::transaction(),
            'consumer_user_id' => $user->id,
            'umkm_product_id' => $product->id,
            'product_name_snapshot' => $product->name,
            'quantity' => $data['quantity'],
            'total_amount' => $data['total_amount'] ?? $this-
>resolveMoney($data['total_label'] ?? null),
            'payment_status' => $data['payment_status'] ?? 'unpaid',
            'status' => ConsumerTransactionStatus::Processing->value,
            'note' => $data['note'] ?? null,
        ]);

        $product->forceFill([
            'stock_qty' => $product->stock_qty - $data['quantity'],
```

```
    ]->save();

    return $transaction;
});

return ApiResponse::created([
    'id' => $transaction->code,
    'status' => $transaction->status,
    'payment_status' => $transaction->payment_status,
], 'Transaksi berhasil dibuat');
}
```